Starting from the upper-left corner (0, 0), find a path to the lower-right corner (N-1, N-1) such that the sum of all cell values along the path is minimized.

From a cell (i, j), you can move only to:

Down: (i+1, j) Right: (i, j+1) Diagonal: (i+1, j+1)

You must print the minimum path cost and the path of cells that gives this minimum cost.

If multiple paths have the same minimum cost, print any one of them.

Input Format

The first line contains an integer T, the number of test cases. For each test case: The first line contains an integer N, the size of the square grid. The next N lines contain N space-separated integers representing the grid.

Constraints

1 ≤ T ≤ 100 1 ≤ N ≤ 500 1 ≤ grid[i][j] ≤ 10^6

Output Format

For each test case:

Print the minimum path as a sequence of cell coordinates in the format (row,column). In the next line, print the minimum path cost.

Sample Input 0

```
3 3
1 2 3
4 8 2
1 5 3
```

Sample Output 0

```
8
```

Sample Input 1

```
2 2
1 2
3 4
```

Sample Output 1

```
5
```

```
r, c = map(int, input().split())

a = [list(map(int, input().split())) for _ in 

dp = [[0] * c for _ in range(r)]
dp[0][0] = a[0][0]

for i in range(r):
    for j in range(c):
        if i == 0 and j == 0:
            continue
        x = []
        if i > 0:
            x.append(dp[i-1][j])
        if j > 0:
            x.append(dp[i][j-1])
        if i > 0 and j > 0:
            x.append(dp[i-1][j-1])
        dp[i][j] = a[i][j] + min(x)
print(dp[r-1][c-1])

```

Line: 21 Col: 1

Upload Code as File

Run Code Submit Code

Test against custom input

Congratulations!

You have passed the sample test cases. Click the submit button to run your code against all the test cases.

Sample Test case 0

Sample Test case 1

Input (stdin) Download

```
3 3
1 2 3
4 8 2
1 5 3
```

Your Output (stdout)

```
8
```

Expected Output Download

```
8
```